DeployHub databases

Which databases DeployHub uses, where each one runs and stores its data, and what is in them · checked on 26 October 2026

> **In short:** DeployHub keeps everything about users, projects and deployments in **one Oracle database** running in Docker on this Mac. Apps you deploy can get their own **MySQL database** in one click, on a shared MySQL server that DeployHub started the first time it was needed. The other databases below are only for development and the automated tests. All of it costs nothing and nothing runs in the cloud.

1. Overview

Database	Used for	Runs where	Data stored in	Status
Oracle Database Free 23.26 ("26ai Free")	DeployHub itself: accounts, projects, deployments, logs, settings	Docker container `deployhub-oracle`, `127.0.0.1:1521`	Docker volume `deployhub-backend_deployhub-oracle-data` (3.3 GB)	**Running**, in use
MySQL 8.4 (shared server)	Databases for the apps you deploy (one per project)	Docker container `deployhub-mysql`, `127.0.0.1:3307`	Docker volume `deployhub-mysql-data` (219 MB)	**Running**, 2 app databases
H2 (file)	Only the `local` profile, for development without Oracle	Inside the backend process	`deployhub-backend/data/deployhub.mv.db` (72 KB, old test data)	Not used now
H2 (in memory)	The automated tests (232)	Inside the test run	Nowhere: gone after each run	Tests only
Oracle in Testcontainers	The Oracle test suite (136)	A temporary Docker container	Nowhere: removed after the run	Tests only

2. Oracle: DeployHub's own database

Setting	Value
Created with	`docker compose -f docker-compose.oracle.yml up -d` (file in deployhub-backend/)
Image	`gvenzl/oracle-free:23-slim-faststart`
Address	`jdbc:oracle:thin:@//localhost:1521/FREEPDB1` (this Mac only, not your network)
App user	`deployhub_app`; its password is in `deployhub-backend/.env` (not shown here)
Used when	the backend runs with the `oracle` profile (how you run it)
Schema	created and upgraded by Flyway; now at version **V9**
Cost	free (Oracle Database Free: up to 2 CPUs, 2 GB RAM, 12 GB data)

Tables and what they hold (rows counted today)

Table	What it stores	Rows
`USERS`	Accounts (email, name, BCrypt password hash, status)	1
`PROJECTS`	Projects: repository, branch, type, commands, port, fixed address	4
`DEPLOYMENTS`	Each deployment: status, commit, image, container, timings	4
`DEPLOYMENT_LOGS`	Log lines of every deployment (secrets masked)	1,877
`DEPLOYMENT_JOBS`	The work queue: attempts, leases, retries	4
`DEPLOYMENT_STATUS_HISTORY`	Every status change: when, why, by whom	16
`IDEMPOTENCY_KEYS`	Stops a double click from creating two deployments	4
`AUDIT_EVENTS`	Security log: sign-ups, logins, changes (no secrets)	54
`PROJECT_ENV_VARS`	Environment variables, values **encrypted**	17
`PROJECT_ROUTES`	Routes such as /api -> another project	2
`PROJECT_DATABASES`	Which project has which MySQL database	2
`DATABASE_SERVERS`	The MySQL server's admin password, **encrypted**	1
`PASSWORD_RESET_CODES`	Forgot password: **hash** of each emailed 6-digit code, expiry, guesses, used (removed with the account)	0
`flyway_schema_history`	Which schema versions are applied	9

Today's contents: one account (SAJAL TOMAR) with four projects: **employeemanagement** (Node.js API) and **employeefrontend** (static site) from `github.com/123sajal/Employee-Management-`, plus **deployhub** and **deployhub-web** (DeployHub deployed inside itself, as a test). Password reset codes are deleted once used or replaced, so that table is usually empty.

Schema versions (Flyway)

Version	Added
V1 init schema	users, projects, deployments, deployment logs
V2 authentication	passwords, account status, audit events
V3 durable deployment jobs	job queue, status history, idempotency keys
V4 rollback	link from a rollback to the deployment it redeploys
V5 real builds	exact commit, image, container, port per deployment; app port and health check
V6 env vars and gateway	environment variables, fixed project addresses, routes
V7 managed databases	MySQL server record and project databases
V8 public links	public link switch and current link per project
V9 password reset	PASSWORD_RESET_CODES: one-time codes for Forgot password

3. MySQL: one-click databases for your apps

Setting	Value
Created	automatically, the first time any project clicks **Add MySQL database**
Image / container	`mysql:8.4` / `deployhub-mysql` (512 MB limit)
Data	Docker volume `deployhub-mysql-data`: survives restarts of the container and your Mac

Setting	Value
Apps connect to	`deployhub-mysql:3306` (private Docker network `deployhub-net`)
You connect to	`127.0.0.1:3307` from this Mac, e.g. `mysql -h 127.0.0.1 -P 3307 -u <user> -p`
Admin password	generated on first use, stored encrypted in DATABASE_SERVERS; never shown
Per project	database and user `p<projectId>_<name>`; the user can only use its own database. Your project would get `p41_employeemanagement`.
App settings	`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DATABASE_URL` are set on the project automatically (encrypted)
Removed when	you click Remove database, or delete the project: the database and all its data go

4. Where passwords and keys are kept

What	Where	Protection
Oracle connection (DB_URL, DB_USERNAME, DB_PASSWORD)	`deployhub-backend/.env`	readable only by you (600), never in git
`DEPLOYHUB_SECRETS_KEY`	`deployhub-backend/.env`	encrypts all stored secrets; **never change or lose it**
Your apps' environment variables	Oracle, PROJECT_ENV_VARS	AES-256-GCM encrypted; never shown again
Each app database's password	only in its project's DB_PASSWORD variable	encrypted; shown once
MySQL admin password	Oracle, DATABASE_SERVERS	encrypted; never shown
Account passwords	Oracle, USERS	BCrypt hashes (can't be turned back into passwords)
Password reset codes	Oracle, PASSWORD_RESET_CODES	BCrypt hashes; work once, 10 minutes, 5 guesses
Gmail App password (MAIL_PASSWORD)	`deployhub-backend/.env`	only if you set up email; never in git

5. Useful commands

To	Run (in Terminal)
Start Oracle after a restart of your Mac	`cd ~/Desktop/DeployHub/deployhub-backend && docker compose -f docker-compose.oracle.yml up -d`
See the database containers	`docker ps --filter name=deployhub-`
See the data volumes and their size	`docker system df -v | grep deployhub`
Stop Oracle (data is kept)	`docker compose -f docker-compose.oracle.yml stop`

> **Never run** `docker compose ... down -v`, `docker volume rm` or `docker volume prune` on these volumes: **there are no backups yet** (you chose to add them with the cloud setup in Milestone 7), so the data would be gone for good.

6. Risks and limits

- **No backups** of Oracle or MySQL yet. A deleted volume or a broken disk loses the data.

- **Losing DEPLOYHUB_SECRETS_KEY** makes stored environment variables and the MySQL admin password unreadable.
- **Memory:** Docker has about 4 GB; Oracle uses about 2 GB, MySQL uses 150-250 MB, and every running app needs some too.
- All apps share one MySQL server: a very busy app can slow the others down.

7. Connecting your app: Employee Management

Your app (`github.com/123sajal/Employee-Management-`) had a setup script that created its own database with a fixed name (`employee_management_db`). On DeployHub, as on Render, Railway or Heroku, the platform creates the database and the app must use the one it is given. Two problems followed:

- Its first lines run `CREATE DATABASE` and `USE employee_management_db`. The app's database user may only use its own database (`p41_employeemanagement`), so this stops with "access denied".
- Its sample data uses plain `INSERT`: a second run fails with "duplicate entry".

Part 1 and Part 2 below fix this. No Terminal is needed: Part 1 is done in the browser on github.com, Part 2 in DeployHub. Afterwards, DeployHub creates the tables automatically on every deploy, and running it again is harmless.

Part 1: three code changes, on github.com

Open the file, click the pencil icon (top right) to edit it, then click **Commit changes**.

Edit 1a: `backend/src/database/schema.sql`: delete the first 5 lines:

```
CREATE DATABASE IF NOT EXISTS employee_management_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE employee_management_db;
```

Edit 1b: same file: the 4 lines starting with `INSERT INTO` become `INSERT IGNORE INTO` (rows that already exist are skipped, so the script can run on every deploy):

```
INSERT IGNORE INTO users (username, email, password, role) VALUES
INSERT IGNORE INTO departments (name, description) VALUES
INSERT IGNORE INTO roles (title, description, department_id) VALUES
INSERT IGNORE INTO employees (employee_id, first_name, ...
```

Edit 2: `backend/src/database/setup.js`: add one line (marked) so the script uses the database DeployHub created; on your own computer the fallback name still applies:

```
  const conn = await mysql.createConnection({
    host:               process.env.DB_HOST     || 'localhost',
    port:               parseInt(process.env.DB_PORT || '3306', 10),
    user:               process.env.DB_USER     || 'root',
    password:           process.env.DB_PASSWORD || '',
    database:           process.env.DB_NAME     || 'employee_management_db',   // <- add this line
    multipleStatements: true,
  });
```

Part 2: settings in DeployHub (http://localhost:4200)

Where	What to do
Project **Database** card	Click **Add MySQL database** (the first time takes 1-2 minutes). Copy the password it shows once, if you want to connect yourself.
Edit project button (top right of the project page; not Settings in the left menu)	Build command: `cd backend && npm ci` Start command: `cd backend && npm run db:setup && npm start` Health check path: `/api/health`
Environment variables	Add `JWT_SECRET` (any long random text). The DB_* variables are already there from the database.
Deploy	Press **Deploy**. The log shows "Database setup complete", "MySQL connected", "Health check passed", then LIVE.

The app's sample login is **admin@ems.com / Admin@123** (from its own schema file). Change it once everything works.

Who has to do Part 1 and Part 2?

Who	Part 1 (edit the code)	Part 2 (settings in DeployHub)
You, once (owner of the code)	Once. After the commit on GitHub the fix stays in the code.	Once for your project. Later deploys are one click.
People who use your app (log in, manage employees)	Nothing.	Nothing: they just open the app's link.
Other DeployHub users deploying their own apps	Only if their app has the same problem. Most apps already read their database settings from environment variables.	Once per project: settings, database, variables.

Why Part 1 is needed once, and only for some apps: creating its own database with a fixed name works on a laptop but not on hosting platforms, which hand the app a ready-made database. Render, Railway and Heroku need the same change. It is a one-time fix that makes the app cloud-ready.

Why Part 2 is normal: every hosting platform asks once per project how to build and start it, which database and which secrets. After that, deploying is one click. (A possible next step for DeployHub: read these settings from a `deployhub.json` file in the repository, like Render's `render.yaml`, so even Part 2 fills itself in.)

Details: README.md · decision records docs/adr/0002 (Oracle), 0007 (secrets), 0008 (one-click databases)